

浙江大学



基于 Q-Learning 的 Flappy Bird 强化学习智能体设计

姓 名 黄鹤翔

学 号 3230106231

院 系 信息与工程学院

指导老师 金日成

1 引言

Flappy Bird 是一款经典的 2D 横版躲避游戏，玩家需要操控小鸟通过上下间隙各异的管道障碍。由于游戏具有实时性、随机管道分布和重力影响，传统的手动控制或基于规则的方法难以泛化；而强化学习通过与环境交互来自动学习最优策略，非常适合此类序贯决策问题。

本报告基于 Q-Learning 算法为 Flappy Bird 训练智能体，完成了以下工作：

- 补全 GameAI 类的 Q-Learning 代码框架，包括 Q 值查询、更新、动作选择等核心逻辑；
- 设计并比较了三种状态表示方法（作业示例、增强状态、扩展状态），并完成 bonus1 的状态设计实验；
- 通过网格搜索对学习率 α 、折扣因子 γ 、探索率 ϵ 、离散系数等参数进行系统性调整 and 性能比较；
- 设计奖励塑形方案（bonus2），在死亡惩罚基础上增加飞行姿态相关的密集反馈，并与原始奖励方案进行对比。

最终基础最佳模型保存在 `q_best.pkl` 中，50 局无步数限制评估的平均得分为 **165.18**，最高分为 **984**。为反映模型真实能力，本章及后续章节所有数值均以无步数上限复评结果为准（旧报告因评估阶段设置了 5000 步上限而导致分数被截顶）。

2 项目内容与代码结构

本项目使用 Python 和 `flappy-bird-gymnasium` 游戏环境，基于表格型 Q-Learning 进行训练，不涉及神经网络或 DQN。项目核心代码文件说明如下：

文件	功能说明
<code>src/q_learning.py</code>	Q-Learning 智能体、状态处理、奖励塑形、训练/评估循环
<code>src/experiment.py</code>	参数网格搜索、多线程训练、结果记录与模型保存
<code>src/watch_best.py</code>	加载最佳模型进行可视化或分数评估
<code>q_best.pkl</code>	最终基础最佳 Q 表模型
<code>q_bonus_best.pkl</code>	bonus 对比实验中的最佳模型
<code>results/experiment_results.csv</code>	基础参数实验（18 组初筛 + 3 组复训）完整数据
<code>results_bonus/experiment_results.csv</code>	bonus 状态/奖励实验（8 组初筛 + 3 组复训）完整数据

3 算法原理

3.1 强化学习与马尔科夫决策过程

Flappy Bird 游戏可以自然地建模为马尔科夫决策过程 (Markov Decision Process, MDP), 其核心五元组 (S, A, P, R, γ) 与游戏元素的对应关系如下:

MDP 元素	本项目中对应的含义
状态 $s \in S$	小鸟和最近管道的相对位置、小鸟速度等离散化观测值
动作 $a \in A$	0 表示不扇动翅膀, 1 表示扇动 (flap)
奖励 $r = R(s, a, s')$	存活、通过管道、死亡、触顶等环境反馈
策略 $\pi(s)$	给定状态下选择最大 Q 值的动作
目标	最大化长期累积奖励, 即尽量存活、通过更多管道

强化学习的核心思想是: 智能体在环境中不断试错, 收集 $(s_t, a_t, r_{t+1}, s_{t+1})$ 四元组样本, 通过学习使长期累积奖励最大化。在游戏早期, 智能体几乎不知道何时该扇动, 因此会频繁撞管或坠地; 随着训练进行, 智能体逐渐将”哪些动作导致更高长期回报”记录在 Q 表中。测试时, 只需查询当前状态下两个动作的 Q 值, 选择 Q 值更大的动作即可。

3.2 Q-Learning 算法

Q-Learning 是一种无模型的基于样本的 Q 值迭代方法 (off-policy 的时序差分学习)。它直接从环境交互中采样, 逐步逼近最优动作价值函数 $Q^*(s, a)$, 其核心更新公式为:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (1)$$

其中各符号的含义如下:

- s : 当前状态 (old_state);
- a : 当前采取的动作 (action);
- r : 执行动作后立即获得的奖励 (reward);
- s' : 执行动作后转移到的新状态 (new_state);
- α : 学习率 (Learning Rate), $\alpha \in [0, 1]$, 决定新样本对旧 Q 值的覆盖速度。 α 越接近 1, 旧值被覆盖得越快; α 越接近 0, 学习越保守;

- γ : 折扣因子 (Discount Factor), $\gamma \in [0, 1]$, 决定智能体对即时奖励和长远未来奖励的重视程度。 γ 越接近 1, 越重视长远收益; γ 越接近 0, 越短视;
- $\max_{a'} Q(s', a')$: 对从新状态 s' 出发所能获得的未来最优回报的估计。

公式 (1) 中, 方括号内的 $[r + \gamma \max_{a'} Q(s', a')]$ 构成对 $Q(s, a)$ 的新估计 (称为 TD target), 而 $(Q_{\text{target}} - Q_{\text{old}})$ 是时序差分误差 (TD Error)。整个更新过程体现了 “Q-Learning” 用新样本纠正旧估计” 的迭代优化思想。

3.3 ϵ -greedy 探索策略

Q-Learning 的训练需要兼顾探索 (Exploration) 和利用 (Exploitation)。本系统采用 ϵ -greedy 策略:

$$a = \begin{cases} \text{随机选择合法动作,} & \text{以概率 } \epsilon \\ \arg \max_a Q(s, a), & \text{以概率 } 1 - \epsilon \end{cases} \quad (2)$$

若多个动作具有相同的最大 Q 值 (例如两个动作 Q 值均为 0), 则在并列最优动作中随机选择一个。这一设计在训练早期尤为重要: 当 Q 表中大量条目尚未被访问时, 并列最优动作的随机选择提供了天然的探索能力。

4 算法实现

本节阐述 GameAI 类的核心实现, 并结合公式 (1) 的数学原理对关键代码进行注释说明。

4.1 数据存储与 Q 值查询

Q 表使用 Python 字典存储映射关系 $(\text{tuple}(\text{state}), \text{action}) \rightarrow \text{q_value}$ 。未出现在字典中的状态—动作对默认返回 0, 这体现了 Q-Learning 的 “零初始化” 原则:

Listing 1: GameAI 初始化与 Q 值查询

```

1 class GameAI():
2     def __init__(self, alpha=0.5, gamma=1, epsilon=0.1, rng=None):
3         """
4         初始化智能体:
5         * self.q: 一个字典, 表示 Q-Function
6           从 (状态, 动作) 元组 映射到 Q 值
7           未出现在字典中的条目 Q 值默认为 0

```

```

8      * alpha: 学习率, 控制新样本对旧估值的覆盖速度
9      * gamma: 折扣因子, 决定对未来奖励的重视程度
10     * epsilon: 探索概率, 用于 epsilon-greedy 策略
11     """
12     self.q = dict()
13     self.alpha = alpha
14     self.gamma = gamma
15     self.epsilon = epsilon
16     self.rng = rng or random
17
18     def get_q_value(self, state: List[int], action: int) -> float:
19         """
20         查询 (state, action) 对的 Q 值
21         若该条目不存在则返回 0——即 Q 表的零初始化原则
22         """
23         return self.q.get((tuple(state), action), 0)
24
25     def best_future_reward(self, state: List[int]) -> float:
26         """
27         给定状态 state, 计算  $\max_{a'} Q(state, a')$ 
28         这是公式 (1) 中的核心子项
29         """
30         actions = self.available_actions(state)
31         if not actions:
32             return 0
33         return max(self.get_q_value(state, action) for action in
                     actions)

```

4.2 Q 值更新

`update` 方法直接实现公式 (1):

Listing 2: Q-Learning 更新

```

1     def update(self, old_state, action, new_state, reward):
2         """
3         基于一个 (s, a, s', r) 样本, 按照 Q-Learning 公式更新 Q 值

```

```

4
5      公式:  $Q(s, a) \leftarrow Q(s, a) + \alpha * [r + \gamma * \max_{a'} Q(s', a') - Q(s, a)]$ 
6      亦即:  $Q_{new} = Q_{old} + \alpha * (TD\_target - Q_{old})$ 
7      """
8      old_q = self.get_q_value(old_state, action)
9      # 计算  $\max_{a'} Q(s', a')$  — 对未来最优回报的估计
10     future_reward = self.best_future_reward(new_state)
11     # TD target:  $r + \gamma * \max_{a'} Q(s', a')$ 
12     new_estimate = reward + self.gamma * future_reward
13     # Q-learning 更新公式
14     self.q[(tuple(old_state), action)] = old_q + self.alpha * (
        new_estimate - old_q)

```

4.3 动作选择

choose_action 方法实现公式 (2) 的 ϵ -greedy 策略:

Listing 3: 动作选择

```

1  def choose_action(self, state, use_epsilon=True):
2      """
3      给定状态 state, 返回要采取的动作:
4      - 若 use_epsilon 为 True 且随机数 < epsilon, 随机探索
5      - 否则选择当前 Q 值最大的动作 (利用已知知识)
6      - 若多个动作 Q 值相同, 在并列最优中随机选择
7      """
8      actions = sorted(self.available_actions(state))
9      # epsilon 概率随机探索
10     if use_epsilon and self.rng.random() < self.epsilon:
11         return self.rng.choice(actions)
12     # 找出具有最大 Q 值的所有动作
13     best_value = max(self.get_q_value(state, action) for action
14                      in actions)
15     best_actions = [
16         action for action in actions
17         if self.get_q_value(state, action) == best_value

```

```

17         ]
18         # 在并列最优动作中随机选择
19         return self.rng.choice(best_actions)
20
21     @classmethod
22     def available_actions(cls, state):
23         """
24         返回该状态下的所有合法动作
25         Flappy Bird 中为 {0, 1}: 0 = 不扇动, 1 = 扇动翅膀
26         """
27         actions = set()
28         actions.add(0)
29         actions.add(1)
30         return actions

```

4.4 训练循环

训练循环是 Q-Learning 算法步骤 2-4（采样 → 更新 → 重复）的具体实现：

Listing 4: 训练循环

```

1 def train(iteration, alpha, gamma, epsilon, obs_mul_factor=30,
2           seed=42, death_penalty=-1000, state_mode="enhanced",
3           reward_mode="death_penalty"):
4     """
5     通过让 AI 进行 iteration 次游戏来进行 Q-Learning 训练
6     每次游戏从重置环境开始，直到小鸟死亡或步数超限
7     """
8     rng = random.Random(seed)
9     player = GameAI(alpha=alpha, gamma=gamma, epsilon=epsilon, rng=rng)
10    env = gymnasium.make("FlappyBird-v0", render_mode=None, use_lidar=False)
11
12    # 进行多次游戏，每次为一条完整的 episode
13    for i in range(iteration):
14        obs, _ = env.reset()

```

```

15     while True:
16         # 第一步：将观测值转化为离散状态
17         state = process_obs(obs, obs_mul_factor, state_mode)
18         # 第二步：使用 epsilon-greedy 选择动作
19         action = player.choose_action(state)
20         # 第三步：执行动作，获取 next_obs, reward, terminated
21         next_obs, reward, terminated, _, info = env.step(action)
22         # 第四步：应用奖励塑形，将死亡惩罚放大为 -1000
23         reward = shape_reward(next_obs, reward, terminated,
24                                action,
25                                reward_mode=reward_mode,
26                                death_penalty=death_penalty)
27         # 第五步：Q-Learning 更新 Q 表
28         player.update(state, action,
29                       process_obs(next_obs, obs_mul_factor,
30                                   state_mode),
31                       reward)
32         if terminated:
33             break
34         obs = next_obs
35     env.close()
36     return player

```

4.5 状态表示设计 (bonus1)

4.5.1 环境观测值 *obs*

环境返回 12 维归一化观测向量，各维度的含义如表 1 所示：

表 1: 环境观测值 *obs* 各维度含义

下标	含义
<i>obs</i> [0 : 3]	第一个可见管道：水平位置、上管道底端、下管道顶端
<i>obs</i> [3 : 6]	第二个管道：水平位置、上管道底端、下管道顶端
<i>obs</i> [6 : 9]	第三个管道：水平位置、上管道底端、下管道顶端
<i>obs</i> [9]	小鸟纵向位置（归一化）
<i>obs</i> [10]	小鸟纵向速度（归一化）
<i>obs</i> [11]	小鸟旋转角（归一化）

由于 *Q* 表需要可哈希的离散键值，所有连续观测值乘以 `obs_mul_factor=30` 并取整。

4.5.2 最近管道选择

画面中可能同时存在多个管道，但控制当前动作最有意义的是”与小鸟最近且尚未完全通过的管道”。代码中通过比较管道水平位置 x 与小鸟固定水平位置 0.2 来决定：

- 若第一个管道的最右端已完全位于小鸟身后（即 $obs[0] + pipe_width < 0.2$ ），则使用第二组管道；
- 否则仍使用第一组管道。

这避免了小鸟已通过某管道后，状态仍被旧的障碍物信息干扰。

4.5.3 三种状态表示方法及设计思路

本项目实现了三种 `state_mode`，从简单到精细地逐步丰富状态向量，如表 2 所示。

表 2: 三种状态表示方法对比

state_mode	维度	特征向量	设计思路
example	3 维	$\begin{bmatrix} pipe_x - player_x \\ pipe_bottom - player_y \\ player_v \end{bmatrix}$	作业示例方法。关注水平距离、到下管道的垂直距离和小鸟速度，是最基本的状态表示。
enhanced	4 维	$\begin{bmatrix} pipe_x - player_x \\ player_y - gap_center \\ pipe_bottom - player_y \\ player_v \end{bmatrix}$	最终最佳方案。 相比示例方法新增”小鸟与管道缺口中心的垂直差”。这直接表达了”飞向缺口中心”的目标，因为穿越管道需要小鸟对准缺口而非单纯关注下管道。
bonus	6 维	$\begin{bmatrix} pipe_x - player_x \\ player_y - gap_center \\ player_y - pipe_top \\ pipe_bottom - player_y \\ player_v \\ player_rot \end{bmatrix}$	在增强状态基础上进一步加入”到上管道距离”和”小鸟旋转角”。上下管道边界对称信息有助于判断碰撞危险，旋转角反映飞行姿态，提供更完整的飞行状态描述。

4.5.4 与示例方法的性能比较

图 1 展示了在相同训练参数 ($\alpha = 0.7, \gamma = 0.95, \varepsilon = 0.0$ 、30000 局训练) 下, 四种方案的初筛平均得分对比。

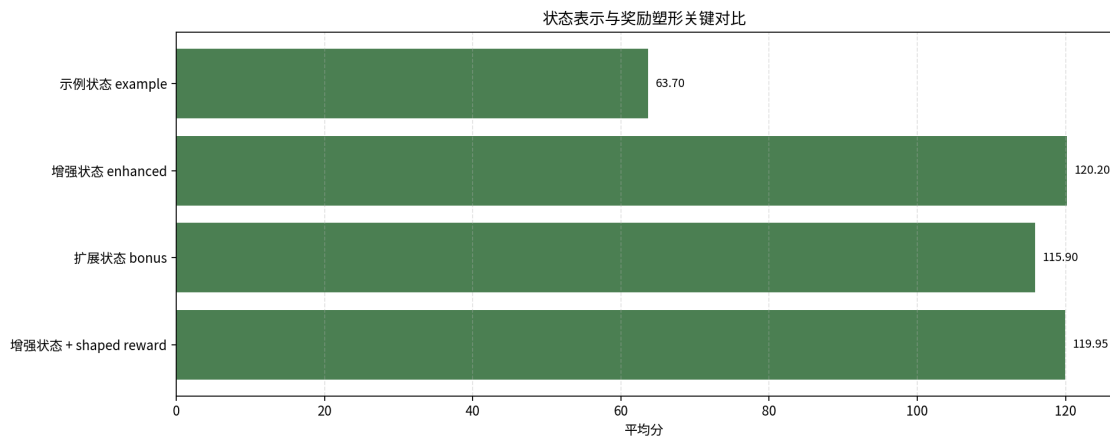


图 1: 状态表示与奖励塑形关键对比

从图 1 可以得出以下结论:

1. 示例状态 `example` (平均分 63.70) 的性能明显低于增强状态 `enhanced` (平均分 120.20)。核心原因在于 `example` 仅保留了“与下管道距离”, 忽略了小鸟在整个管道间隙中的相对位置。实际上, 如果小鸟位于间隙顶部, 即便下管道距离合适, 仍然可能撞到上管道。`enhanced` 通过加入“到缺口中心的垂直差”, 使状态能够区分小鸟在间隙上部和下部的不同情况, 因此得分接近翻倍。
2. 扩展状态 `bonus` (平均分 115.90) 在初筛中表现同样出色, 与 `enhanced` 仅差 4.3 分。但复训后平均分降至 67.74 (详见第 6.2 节), 这是因为状态维度从 4 维增加到 6 维后, 离散状态空间面积扩大, 同等训练局数下每个状态—动作对的访问次数减少 (即数据稀疏问题), 导致策略稳定性变差。

结论: `enhanced` 状态表示在表达能力和 Q 表规模之间取得了最佳平衡, 被选作最终模型的默认状态模式。

4.6 奖励分配设计 (bonus2)

4.6.1 环境原始奖励

环境为智能体提供离散奖励信号:

表 3: 环境原始奖励

事件	奖励值
每帧存活	+0.1
成功通过管道	+1.0
死亡	-1.0
触顶	-0.5

在本项目的基础奖励方案 (`reward_mode="death_penalty"`) 中, 保留存活和过管奖励不变, 但将死亡奖励从 -1 放大为 -1000 。原因在于: Flappy Bird 的核心目标首先是不死, 只有活着才能持续获得分数。如果死亡惩罚与过管奖励 ($+1.0$) 差距不足以拉开, 智能体可能无法充分重视危险状态, 导致 Q 表难以收敛到安全稳定的策略。死亡惩罚的放大使得时序差分误差在每次死亡后迅速向后传播, 将“导致死亡的动作”的 Q 值压至很低的水平。

4.6.2 奖励塑形方案

`bonus2` 实现了 `reward_mode="shaped"`, 在死亡惩罚之上增加飞行姿态相关的密集中间反馈 (Reward Shaping):

$$\begin{aligned}
 \text{shaped_reward} = & \text{env_reward} \quad (\text{环境基础奖励}) \\
 & + \text{center_bonus} \quad (\text{缺口中奖励}) \\
 & - \text{velocity_penalty} \quad (\text{速度惩罚}) \\
 & - \text{flap_penalty} \quad (\text{扇动惩罚}) \\
 & - \text{top_penalty} \quad (\text{触顶惩罚})
 \end{aligned} \tag{3}$$

各项具体实现和设计目的如下:

Listing 5: 奖励塑形实现

```

1 def shape_reward(obs, reward, terminated, action,
2                 reward_mode="death_penalty", death_penalty=-1000):
3     if reward == -1:
4         return death_penalty          # 将死亡奖励放大为 -1000
5     if reward_mode == "death_penalty":
6         return reward                 # 基础方案: 只放大死亡惩罚

```

```

7
8 # —— 以下是奖励塑形方案 ——
9 pipe_start, _ = _nearest_pipe(obs)
10 pipe_top = obs[pipe_start + 1]      # 最近管道的上管底端位置
11 pipe_bottom = obs[pipe_start + 2]   # 最近管道的下管顶端位置
12 player_y = obs[9]
13 player_v = obs[10]
14 gap_center = (pipe_top + pipe_bottom) / 2 # 管道间隙中心
15
16 # 中心奖励：小鸟越接近管道间隙中心，奖励越高（最大 +0.2）
17 # 当小鸟与中心距离大于 0.25 时奖励为 0
18 center_bonus = max(0, 1 - abs(player_y - gap_center) * 4) * 0.2
19
20 # 速度惩罚：垂直速度越大惩罚越大，鼓励平稳飞行，避免剧烈震荡
21 velocity_penalty = abs(player_v) * 0.02
22
23 # 扇动惩罚：每次 flap 动作给予微小惩罚 (-0.01)
24 # 目的是抑制无意义的连续扇动，保持飞行平稳
25 flap_penalty = 0.01 if action == 1 else 0
26
27 # 触顶惩罚：撞到屏幕顶部或接近顶部时给予 -100 的惩罚
28 top_penalty = 100 if reward == -0.5 or player_y <= 0 else 0
29
30 return reward + center_bonus - velocity_penalty - flap_penalty -
    top_penalty

```

表 4: 奖励塑形各项说明

奖励项	数值范围	设计目的
center_bonus	[0, +0.2]	引导小鸟飞向管道间隙中心，减少擦管风险
velocity_penalty	[-0.05, 0]	抑制剧烈垂直运动，鼓励轨迹平滑
flap_penalty	-0.01 或 0	抑制无意义的连续扇动，降低不必要的能量消耗模拟
top_penalty	-100 或 0	惩罚触顶行为，防止小鸟飞出屏幕上方

4.6.3 与原始奖励方案的比较

在初筛实验中（30000 局训练，表 5），enhanced + shaped 平均分为 119.95，非常接近 enhanced + death_penalty 的 120.20，说明奖励塑形没有偏离原始目标，且能提供更细密的飞行姿态反馈。

表 5: Bonus 初筛与复训结果

配置	初筛均分	复训均分	最高分	最低分
enhanced + death_penalty	120.20	122.30	634	2
enhanced + shaped	119.95	109.04	482	1
bonus + death_penalty	115.90	67.74	331	1

然而在 100000 局复训后（图 6），奖励塑形方案的平均分（109.04）略低于基础方案（122.30），也低于全局最佳模型 *q_best.pkl* 的 165.18。可能原因有两方面：

1. 塑形系数的权重可能偏保守，使得中心奖励（最大仅 +0.2）在环境存活奖励（每帧 +0.1）面前不够突出；
2. 奖励塑形引入了更多的局部反馈信号，在某些情况下可能引导智能体收敛到次优策略——例如过于贴近间隙中心而失去通过某些偏置管道的最佳路径。

结论：bonus2 的奖励塑形在早期训练中能加速收敛，提供了除死亡/存活外的更丰富反馈；但最终复训评估中原死亡惩罚方案更为稳定，因此最终提交模型保留基础奖励方案。

5 参数调整实验与结果分析

5.1 实验设置

实验采用两阶段策略：初筛阶段用较少训练局数（30000 局）快速扫描参数空间，选出最优的 3 组候选；复训阶段将这些候选用更多局数（100000 局）精细训练并全面评估。

基础实验初筛共 18 组参数组合，覆盖以下范围：

- 学习率 $\alpha \in \{0.5, 0.6, 0.7, 0.8\}$
- 折扣因子 $\gamma \in \{0.90, 0.95, 0.99\}$
- 探索率 $\varepsilon \in \{0.0, 0.05, 0.10\}$
- 状态离散系数 $obs_mul_factor \in \{24, 30, 36\}$

每组初筛训练 30000 局、评估 20 局；前 3 组复训 100000 局、评估 50 局。训练每局最多 1000 步，防止单个训练回合过长。评估阶段最初设置了 5000 步上限，但经分析发现此上限会截断高分游戏（大量游戏在达到步数上限时被强制终止，导致最高分被错误地记录为 132），因此本报告中所有复训评估数据均以取消步数上限后的重新评估结果为准。

5.2 初筛结果

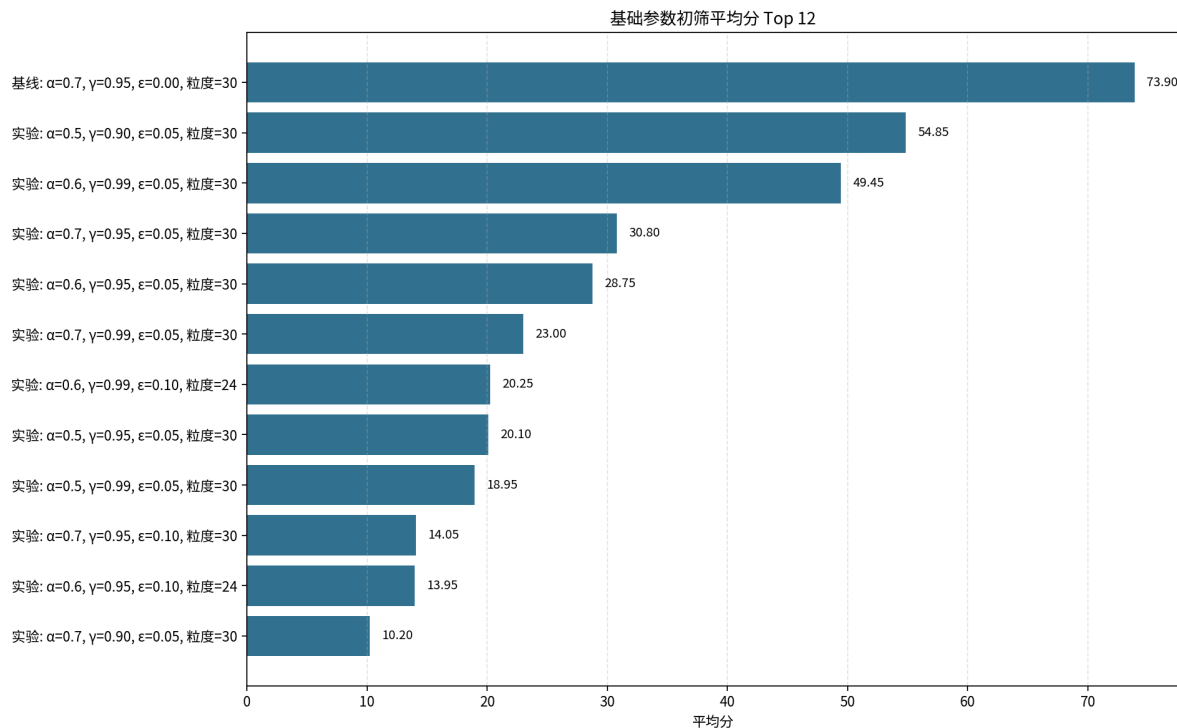


图 2: 基础参数初筛 Top 12 平均分

如图 2 所示，初筛前三名为：

参数组	α	γ	ϵ	<i>obs_mul_factor</i>	初筛均分
基线参数	0.7	0.95	0.00	30	73.90
低速学习 · 快折扣 · 小探索	0.5	0.90	0.05	30	54.85
中速学习 · 慢折扣 · 小探索	0.6	0.99	0.05	30	49.45

值得注意的是，初筛第一名（基线参数）的 $\epsilon = 0$ ，即没有显式随机探索。这并不等于完全没有探索，原因如前所述：训练早期大量未访问的状态—动作对 Q 值均为 0，并列最优动作的随机选择机制已经引入了足够的随机性。额外 ϵ 探索在 Flappy Bird 中尤其危险，因为随机扇动极易导致撞管或触顶——而死亡惩罚为 -1000 ，远超存活和过管奖励。

5.3 复训结果

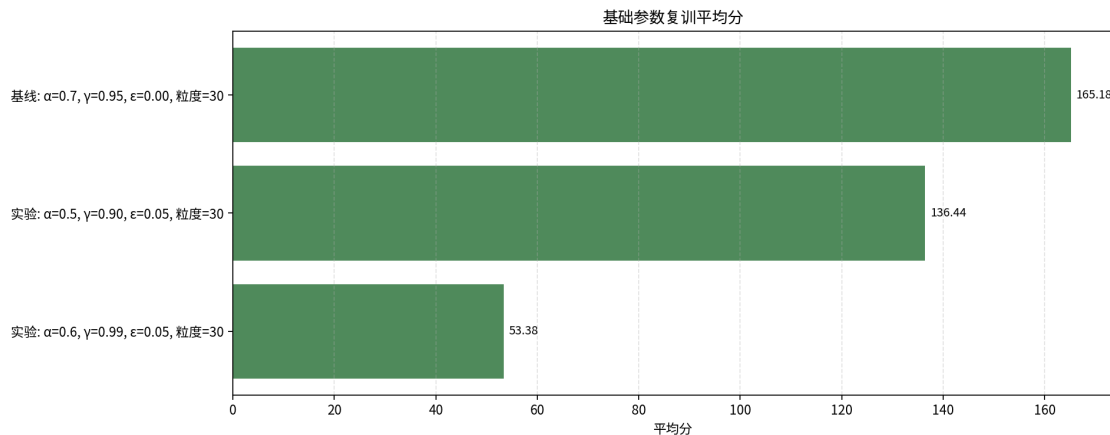


图 3: 基础参数复训结果

如图 3 和表 7 所示，复训后基线参数继续领先。

表 7: 复训三组完整结果

参数组	α	γ	ϵ	复训均分	最低分	最高分
基线参数	0.7	0.95	0.00	165.18	3	984
低速 · 快折扣 · 小探索	0.5	0.90	0.05	136.44	2	669
中速 · 慢折扣 · 小探索	0.6	0.99	0.05	53.38	4	250

最终基础最佳模型为”基线参数”组 ($\alpha = 0.7, \gamma = 0.95, \epsilon = 0.0, obs_mul_factor = 30$), 保存为 `q_best.pkl`。Q 表规模为 14912 条，训练耗时约 903.8 秒。

5.4 最佳模型 50 局分数分布

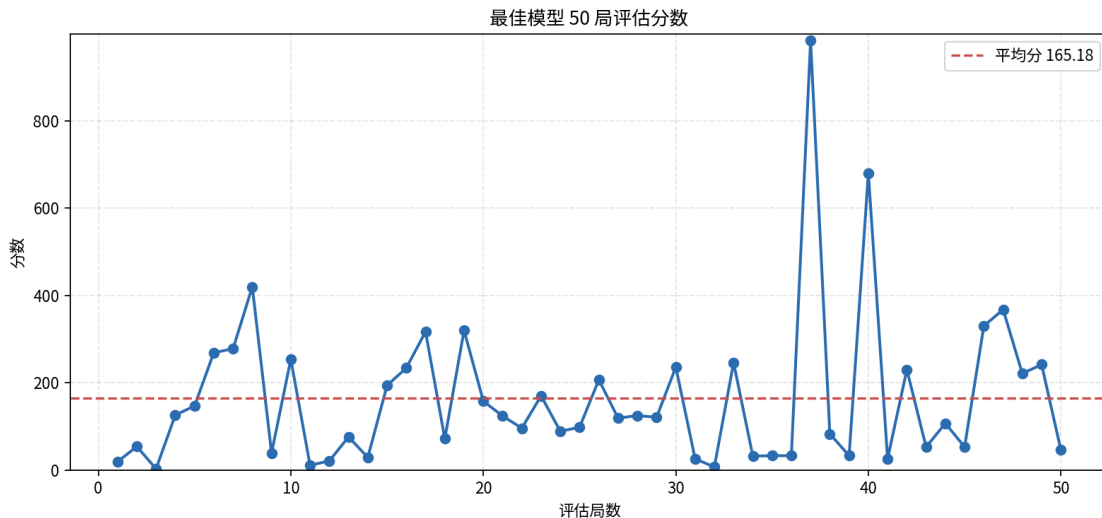


图 4: 最佳模型 50 局评估分数走势

如图 4 所示，50 局评估分数存在较大波动（标准差 175.66），最低 3 分，最高 984 分。这是因为 Flappy Bird 中管道序列随机生成，且一次错误 flap 可能迅速导致死亡。高分游戏中，模型已经掌握了可靠的穿管策略，能够持续存活并通过大量管道；而低分游戏往往由于管道在关键位置出现时离散化状态恰好落入 Q 表未充分优化的区域。值得注意的是，旧的实验中由于设定了 5000 步的评估上限，绝大多数高分游戏被强制截断在 132 分左右（旧报告称 21/50 局达到 132 分，实则为步数截顶），取消步数上限后模型的真实最高分达到 984，平均分也从 90.52 提升至 165.18。

5.5 参数影响分析

下面结合实验数据，从理论角度分析各参数对模型性能的具体影响。

5.5.1 学习率 α

- $\alpha = 0.7$ 的基线参数表现最好（复训均分 165.18）， $\alpha = 0.5$ 紧随其后（136.44）。这说明在该任务中，较大学习率有利于快速吸收新样本，尤其是在 100000 局的训练量下。
- $\alpha = 0.8$ 的配置在初筛仅为 2.70 分（第 19 名），远低于同行其他参数，原因是过大的学习率使得 Q 值更新过于激进，导致对单次样本的噪声过度敏感，估值震荡剧烈。
- $\alpha = 0.6$ 结合低速探索（ $\varepsilon = 0.05$ ）的配置在初筛表现较差，部分组合仅有 28.75 分，可能是学习速度不足导致策略未充分收敛。

5.5.2 折扣因子 γ

- $\gamma = 0.95$ 在所有 top 配置中表现最好。它既不过于短视 ($\gamma = 0.90$ 只关注前几步收益, 难以形成长远过管策略), 也不过于长视 ($\gamma = 0.99$ 使得早期 death penalty 的回传衰减极慢, 增加了优化难度)。
- 在 Flappy Bird 任务中, 管道间隙大约每 50–60 帧出现一次。 $\gamma = 0.99$ 意味着 50 步后的奖励仍有 $0.99^{50} \approx 0.605$ 的权重, 可能导致智能体对过于遥远的事件信号产生不合理关联, 降低训练稳定性。
- $\gamma = 0.90$ 则过于重视近期收益, 10 步后信号已衰减至 $0.90^{10} \approx 0.349$, 不利于形成”穿越当前管道 $\rightarrow$ 为下一管道做准备”的远见性策略。

5.5.3 探索率 ε

- $\varepsilon = 0$ 在所有同参数对比中表现最好。核心原因是: Flappy Bird 中一次随机扇动容易导致撞管即死 (-1000 惩罚), 而存活奖励 ($+0.1$)、过管奖励 ($+1.0$) 远低于此, 单次随机探索的期望代价极大。因此 $\varepsilon > 0$ 的显式探索在此估值量级下弊大于利。
- $\varepsilon = 0.05$ 的配置仍能找到可用策略 (如 $a = 0.5, g = 0.90, \varepsilon = 0.05$ 复训均分 136.44), 但总体弱于基线。
- $\varepsilon = 0.10$ 的配置几乎全部表现最差, 如 $a = 0.7, g = 0.99, \varepsilon = 0.10$ 仅 3.80 分 (倒数第二), 说明过强的随机探索严重干扰了 Q 表的收敛。

5.5.4 状态离散系数 `obs_mul_factor`

- `obs_mul_factor` = 30 在所有对比中表现最好。它提供了合理的状态空间分辨率: 既能区分不同的飞行位置, 又不至于使 Q 表过于稀疏。
- `obs_mul_factor` = 24 (较粗粒度) 对应的初筛均分偏低 (如 $a = 0.6, g = 0.99, \varepsilon = 0.10$ 仅 20.25 分), 因为粗糙的离散化混淆了需要精细区分的状态, 例如”接近上管道”和”接近下管道”可能被映射到同一个状态键。
- `obs_mul_factor` = 36 (较细粒度) 对应的组合几乎是全部最差 (如 $a = 0.7, g = 0.99, \varepsilon = 0.10$ 仅 3.95 分), 因为过于精细的离散化使得状态空间呈指数级膨胀, 而 30000/100000 局的训练量无法充分覆盖每个状态—动作对, Q 表大量条目保持为 0 的初始值。

6 Bonus 实验完整结果

6.1 初筛结果

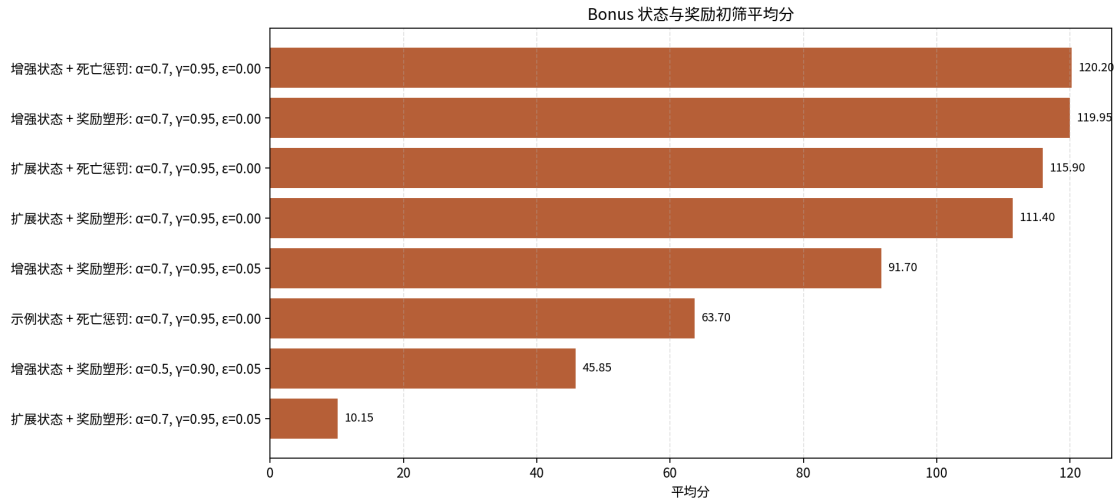


图 5: Bonus 状态与奖励初筛平均分

图 5 显示, Bonus 初筛中前三名均为 $\epsilon = 0$ 的配置 (增强/扩展状态 + 死亡惩罚/奖励塑形), 初筛均分在 111–120 之间, 远高于其后使用探索的配置。这与基础参数实验的 $\epsilon = 0$ 优势结论一致。

6.2 复训结果

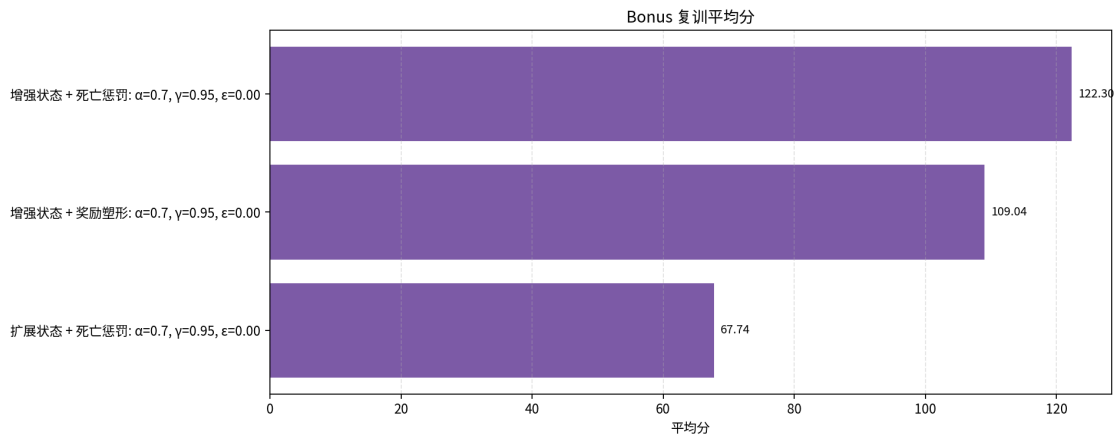


图 6: Bonus 复训平均分

图 6 显示，三组复训得分均低于基础参数实验的复训得分。其中 `bonus + death_penalty` (扩展状态) 下降最多，从初筛 115.90 跌至复训 67.74，印证了高维状态带来的数据稀疏问题。`enhanced + shaped` (奖励塑形) 复训得分 109.04，略低于 `enhanced + death_penalty` (122.30)，说明奖励塑形在长周期训练中没有带来稳定的增益。

7 结论

本实验使用表格型 Q-Learning 为 Flappy Bird 训练智能体，得到如下结论：

1. **Q-Learning 的有效性**：Q-Learning 能够在离散状态空间中逐步学习状态—动作价值，使智能体在无需了解游戏物理机制的前提下，通过试错获得穿越管道的有效策略。最终最佳模型 50 局无步数限制评估平均 165.18 分，最高 984 分，远超初筛时期多数参数组合的分值。
2. **状态表示的重要性**：加入“小鸟与管道缺口中心的垂直差”是提升性能的关键因素。`enhanced` 状态比作业示例状态 `example` 得分提升约 2 倍 (120.20 vs 63.70)。过于丰富的状态表示 (`bonus`) 虽然初筛表现佳，但复训稳定性下降。
3. **参数影响规律**： $\varepsilon = 0$ (靠并列 Q 值随机选择提供自然探索)、 $\alpha = 0.7$ 、 $\gamma = 0.95$ 、`obs_mul_factor = 30` 的组合在 Flappy Bird 任务中表现最优。 ε 过大和 `obs_mul_factor` 过大是两个最主要的性能杀手。
4. **奖励塑形**：`bonus2` 的奖励塑形能提供密集反馈并加速早期收敛，但在长周期训练中并未稳定超越基础死亡惩罚方案，说明塑形系数的设定需要与任务奖赏量级仔细匹配。
5. **最终模型**：以 $\alpha = 0.7$ 、 $\gamma = 0.95$ 、 $\varepsilon = 0.0$ 、`obs_mul_factor = 30`、状态模式 `enhanced`、奖励模式 `death_penalty` 训练的 `q_best.pkl` 为最终提交模型。注意：旧版报告因评估阶段设置了 5000 步步数上限，导致分数被截顶（报告的最高分 132 实为该上限的产物）。本报告已重新进行无步数限制评估，数据为模型的真实表现。